



12. Priority Queues

"Not everyone waits equally. Some tasks simply cannot wait."

Previous Section:

 [11. Trees](#)

Contents:

- [1. What is a Priority Queue?](#)
- [2. Properties of Priority Queues](#)
- [3. Operations on a Priority Queue](#)
- [4. Applications of Priority Queues](#)
- [5. Implementations of Priority Queues](#)

[Summary](#)

[References](#)

Think about a normal queue. You arrive at a movie theater, take a ticket, and wait for your turn. The first person who enters the line is usually the first person served. This is the idea behind a regular queue: **First In, First Out (FIFO)**.

But now imagine a completely different situation. You are managing a hospital. Five patients arrive:

Patient	Age
John	35
Sarah	4
David	68
Emily	8
Alex	15

Should the doctor simply treat them in the order they arrived? Probably not. A toddler may need immediate attention. An elderly patient may require urgent care. A normal adult with a minor issue can probably wait a little longer.

So instead of serving people based solely on arrival time, we assign **priority levels**. Suppose the hospital uses the following priority system:

Priority	Group
1	Toddlers (3–5 years old)
2	Children (6–10 years old)
3	Elders (60+ years old)
4	Adolescents (11–18 years old)
5	Normal Patients

One simple way to represent this is by attaching a letter to the waiting number:

Group	Prefix
Toddlers	A
Children	B
Elders	C
Adolescents	D
Normal Patients	E

Examples:

```
A001 Toddler
A002 Toddler

B001 Child
B002 Child

C001 Elder

D001 Adolescent

E001 Normal Patient
```

Now suppose patients arrive in this order:

```
E001 D001 A001 C001 B001
```

A Priority Queue automatically rearranges them according to priority:

```
Front
|
A001 B001 C001 D001 E001 - Rear
```

Notice something important. The queue does **not** care only about arrival order. It cares about **priority first**, then arrival order within the same priority level. This special behavior is exactly why Priority Queues are used in operating systems, hospitals, network routing, and many Artificial Intelligence applications.

1. What is a Priority Queue?

A **Priority Queue** is a queue in which every element is assigned a priority value. Unlike a normal queue, elements are not always removed in the order they were inserted. Instead, the element with the highest priority is removed first.

Think of it as a queue where importance matters more than arrival time. For example:

```
Inserted:

E001 A001 D001 C001 B001

Priority Queue View:

Front
|
A001 B001 C001 D001 E001 - Rear
```

When removing elements. The highest-priority patients leave the queue first.

```
Remove A001
Remove B001
Remove C001
Remove D001
Remove E001
```

2. Properties of Priority Queues

A Priority Queue stores two pieces of information: **(priority, value)**. Example: (1, A001); (2, B001); (3, C001); (4, D001); (5, E001)

When inserting, we place according to priority. When deleting, we remove highest-priority element first.

Normal Queue:

Front

|

A B C D - Rear

Removal Order: A → B → C → D

Priority Queue:

Priority

1 A

2 B

5 E

3 C

4 D

Removal Order: A → B → C → D → E regardless of insertion order.

Priority Queue and Sorting

An interesting observation is that repeatedly removing elements from a Priority Queue naturally produces a sorted sequence. Example:

Insert:

45 12 78 23 31

Removing continuously:

12 23 31 45 78

(or the reverse order depending on the implementation)

This behavior is closely related to **Heap Sort**, which is why heaps are commonly used to implement Priority Queues.

3. Operations on a Priority Queue

Like other data structures, Priority Queues support several operations.

- Insert (Enqueue): Add a new element according to its priority.

Current Queue

A001 B001 C001

Insert A002:

A001 A002 B001 C001

The new patient joins the end of its own priority group.

- Remove (Dequeue): Remove the highest-priority element.

Before:

A001 A002 B001 C001

After Removal:

A002 B001 C001

- Peek: Return the highest-priority element without removing it.

Queue:

A001 B001 C001

Peek: A001

Queue remains unchanged.

- Traverse: Visit every element in the queue.

Traversal: A001 B001 C001 D001 E001

4. Applications of Priority Queues

Priority Queues appear in many real-world systems because not all tasks are equally important.

- Hospital Management: Patients are treated based on urgency rather than arrival order. **For instance: Toddler → Child → Elder → Adolescent → Normal**

- Operating Systems: The CPU decides which process should execute next.
For instance: High Priority Process → Medium Priority Process → Low Priority Process
- Network Routing. Routers prioritize important packets over less important traffic.
- Artificial Intelligence: Many AI search algorithms explore the most promising states first.
 - Dijkstra's Shortest Path Algorithm: The next node with the smallest distance is always selected first.
 - Prim's Minimum Spanning Tree Algorithm: The cheapest edge is selected repeatedly.
 - Huffman Coding: Characters with smaller frequencies are processed first to create efficient compression trees.

5. Implementations of Priority Queues

A Priority Queue can be implemented using several data structures:

Implementation	Insertion Time Complexity	Deletion Time Complexity
Array	$O(n)$	$O(1)$
Linked List	$O(n)$	$O(1)$
Binary Search Tree (BST)	$O(\log n)$	$O(\log n)$
Heap	$O(\log n)$	$O(\log n)$



Note: The actual performance of a Binary Search Tree depends on its balance. A balanced BST achieves $O(\log n)$ operations, while an unbalanced BST can degrade to $O(n)$ in the worst case. Heaps provide guaranteed $O(\log n)$ insertion and deletion, making them the most common implementation of Priority Queues.

Among these implementations, Heap is the most popular because it provides efficient insertion and deletion operations. This is why most programming languages implement Priority Queues using heaps internally.

Summary

In this section, we explored **Priority Queues**, a special type of queue where elements are processed according to their priority rather than purely by arrival order.

- We began with a hospital management scenario and discovered that some tasks simply cannot wait. By assigning priority levels to elements, a Priority Queue ensures that the most important items are always processed first while still preserving insertion order among elements of the same priority.
- We then studied the fundamental operations of a Priority Queue, including insertion, deletion, traversal, and peeking at the highest-priority element. We also saw how repeatedly removing elements naturally produces a sorted sequence, making Priority Queues closely related to sorting algorithms.

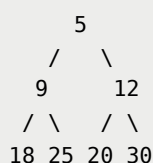
Finally, we examined several real-world applications such as hospital systems, operating systems, network routing, Artificial Intelligence, Dijkstra's algorithm, Prim's algorithm, and Huffman coding. Among all possible implementations, the Heap stands out as the most efficient and widely used structure for Priority Queues.

Heap — The Engine Behind Priority Queues

Throughout this section, we treated the Priority Queue like a magical structure that somehow keeps the highest-priority element at the front. But how does it actually do that efficiently? If thousands—or even millions—of elements are inserted, continuously sorting the entire queue would be extremely expensive.

This is where the **Heap** comes in. A Heap is a specialized tree structure designed to keep the largest or smallest element easily accessible while maintaining efficient insertion and deletion operations.

For example, a Min Heap might look like:



Notice how the smallest value always stays at the root. This simple property allows Priority Queues to retrieve the highest-priority element in an extremely efficient manner.

In the next section, we will dive into **Heaps**, learn how they work internally, and discover why they are considered the backbone of modern Priority Queue implementations.

References

https://drive.google.com/file/d/1VfNgqFPkvnDGZeVKN_QqIHjWCDQRapBM/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/priority-queue-set-1-introduction/>

<https://www.programiz.com/dsa/priority-queue>

Next Section:

 [12.1. Heaps](#)